Predator/Prey Simulation - Design Pattern Diagram: Strategy, Template Method and Static Factory Methods

Strategy pattern participants

Supporting classes

Orange text = Template Method steps

Entity

-x: int
-y: int
-isFood: boolean

+getX(): int / +getY(): int
+setX(int) / +setY(int)
+isFood(): boolean

Grass

-growthTimer: int
-isEaten: boolean

+isEdible(): boolean
+setEaten(boolean)
+tickGrowth()

«abstract»
«Context»
Creature

-speed: int
-starvation: int
-alive: boolean
-strategy: MovementStrategy

+starve(): boolean {template method}
#getStarvationRate(): int {abstract step}
#chooseStrategy()
#setStrategy(MovementStrategy)
#applyStrategy() -> strategy.move(this)
#mutatedSpeed(): int
-metabolicAdjustment(): int
+eat()
+reproduce()

strategy

«interface»
«Strategy»
MovementStrategy

+move(self: Creature)

«creates» lambdas implementing MovementStrategy

«ConcreteStrategy factory»
MovementStrategies

+wander(): MovementStrategy
+idle(): MovementStrategy
+seek(Entity): MovementStrategy
+flee(Creature, double): MovementStrategy
+pursue(Creature, double): MovementStrategy
+herd(List<? extends Creature>, int): MovementStrategy

«Client»
Predator

-sim: List<Creature>

#chooseStrategy()
#getStarvationRate(): int
+eat()
+reproduce()

«Client»
Prey

-inDanger: boolean
-sim: List<Creature>
-grassList: List<Grass>

#chooseStrategy()
#getStarvationRate(): int
+eat()
+reproduce()

uses: picks a strategy each tick

Static factory methods (supporting pattern)

MovementStrategies creates every strategy through a static method, so Predator and Prey never see how a strategy is built. Each method returns a lambda that captures its target.

```
setStrategy(MovementStrategies.pursue(prey, HUNT_SPEED_MULTIPLIER));
```

Strategy - roles

Strategy: MovementStrategy, a functional interface with one method, move(Creature).

Concrete strategies: lambdas returned by MovementStrategies (wander, idle, seek, flee, pursue, herd). Each captures its own target, so no class per behaviour is needed.

Context: Creature holds the current strategy; applyStrategy() delegates to it.

Clients: Predator and Prey choose a strategy at runtime in chooseStrategy().

```
setStrategy(MovementStrategies.flee(threat, FLEE_SPEED_MULTIPLIER));
applyStrategy();
```

Template Method (secondary pattern)

Creature.starve() fixes the algorithm (add the rate plus a metabolic adjustment, then test the threshold) and defers getStarvationRate() to subclasses: Predator returns 3; Prey returns 1, doubled while fleeing.

Creature also declares chooseStrategy(), eat() and reproduce() as abstract steps that Predator and Prey implement.

Game loop (Simulation uses Creature)

Phase 1: snapshot.parallelStream().forEach(Creature::chooseStrategy). Decisions only read shared state and write the creature's own fields.

Phase 2 (sequential): applyStrategy() and eat(), so all world changes stay on one thread.